

2. 文档处理流程

文档处理流程负责把外部知识源转化为 OpenViking 内部可检索、可治理的知识树。该流程采用同步接收、异步语义加工的架构：用户上传或指定远程资源后，系统先完成格式解析与临时目录构建，再通过队列完成摘要、向量化和索引更新。

```
用户/系统提交资源
-> /api/v1/resources 或 Local Client add_resource()
-> ResourceService 注册知识文档和任务状态
-> Parser Registry 选择解析器
-> Parser 在临时 VikingFS 中生成目录树和 L2 内容
-> TreeBuilder 确定最终 Viking URI
-> SemanticQueue 投递 SemanticMsg
-> SemanticProcessor 自底向上生成 L0/L1
-> VectorDB 写入语义索引
-> KnowledgeDocumentRegistry 更新状态
```

2.1 资源接入

- HTTP 接入：/api/v1/resources 支持 path、temp_file_id、to、parent、folder_path、instruction、wait、timeout、include/exclude、watch_interval 等参数。
- 临时上传：/api/v1/resources/temp_upload 先写入上传目录，再由 temp_file_id 转为可处理的本地临时文件。
- 本地接入：Python Local Client 和 Sync/Async HTTP Client 对上层业务隐藏路由细节，常用方法包括 add_resource、add_skill、wait_processed、find、search、read、abstract、overview。
- 远程资源保护：HTTP 模式默认要求远程资源来源，只有 temp_file_id 场景允许解析上传后的本地临时路径，降低服务端任意文件读取风险。

输入类型	解析结果	说明
Markdown / TXT / HTML	保持标题层级、正文块、链接和基础结构	适合作为知识库基础文档
PDF / Word / Excel / PowerPoint	转换为可读文本、表格或页面结构	适合制度、方案、投标资料、产品手册
代码仓库 / 目录 / ZIP	保留目录结构，按文件类型识别代码与文档	支持源码知识库、SDK 文档、工程经验沉淀
图片 / 音频 / 视频	生成多模态摘要并映射到资源目录	用于截图、录屏、会议资料等非纯文本信息
飞书 / EPUB / 旧版文档	通过专用解析器或转换器进入统一树结构	便于企业已有资料迁移

2.2 Parser 与切分策略

Parser 的职责是格式转换和结构化切分，不承担 LLM 摘要生成。这个边界很重要：解析阶段尽可能确定、可重试、低成本；语义阶段再异步调用模型，避免大文件入库阻塞和内存压力。

1. Parser Registry 根据文件扩展名、资源类型和参数选择解析器，例如 markdown.py、pdf.py、word.py、excel.py、powerpoint.py、directory.py、zip_parser.py、repository 相关解析逻辑等。
2. 解析器把原始资料写入临时 VikingFS，形成一个文档根目录，目录内包含可读 L2 内容文件和必要的结构文件。
3. 长文档按标题、章节、token 预算和文件类型切分；小片段会合并，过大片段会继续拆分，目标是在保持语义完整性的同时控制后续摘要和向量化成本。